

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчет о программном проекте на тему:

**Разработка Retrieval-Augmented системы
для справочного чат-бота образовательной организации**

Выполнил:

студент группы БПМИ2411

Я. Густов

(дата)

Принял руководитель проекта:

Д. Андреева

руководитель проекта

Факультета компьютерных наук НИУ ВШЭ

Москва, 2026

Содержание

1	Введение	2
2	Постановка задачи и design doc	2
3	Обзор литературы	2
4	Архитектура системы	3
5	Датасеты и метрики	3
5.1	Retrieval-набор	3
5.2	Generation-набор	3
5.3	Retrieval-метрики	4
5.4	Generation-метрики	4
6	Эксперименты и результаты	4
6.1	Предобработка	4
6.2	Размер чанка и метод чанкирования	4
6.3	Сравнение embedders	5
6.4	Dense, sparse, ensemble и reranking	6
6.5	Финальная проверка на test	6
6.6	Generation	6
6.7	Pilot AgenticRAG	7
7	Обсуждение результатов	7
8	Заключение	7
9	Перспективы дальнейшей разработки	8
10	Использование ИИ	8

1. Введение

Цифровой справочный сервис для образовательной организации должен одновременно быть быстрым, понятным и фактически точным. Для нормативных вопросов это особенно важно: пользователь ожидает краткий ответ на естественном языке, но при этом ответ должен опираться на официальный документ, а не на вероятностную память языковой модели. Поэтому в проекте была выбрана архитектура Retrieval-Augmented Generation (RAG), где генерация ответа отделена от поиска источника, а качество retrieval и generation оценивается отдельно.

В рамках работы был реализован чат-бот по нормативным документам НИУ ВШЭ, включающий конвертацию документов в markdown, предобработку, чанкирование, dense и sparse retrieval, reranking, генерацию ответа и Telegram-интерфейс.

2. Постановка задачи и design doc

Целевой пользователь системы --- студент или абитуриент, которому нужно быстро найти ответ по длинному нормативному документу без ручного чтения pdf/docx-файла. Бизнес-цели проекта состоят в сокращении времени поиска ответа, снижении нагрузки на сотрудников, отвечающих на типовые вопросы, и повышении прозрачности ответа за счёт явной привязки к источнику.

Технические цели проекта:

1. построить воспроизводимый RAG-конвейер для документов ВШЭ;
2. обеспечить устойчивый retrieval на точных, переформулированных и шумных запросах;
3. отдельно оценивать retrieval и generation;
4. подготовить инфраструктуру для сравнения предобработки, чанкирования, embedders, sparse/dense retrieval и reranking;
5. проверить, даёт ли agentic-подход выигрыш на пограничных вопросах.

3. Обзор литературы

RAG-подходы позволяют опираться на внешний корпус документов и тем самым снижать зависимость ответа от параметрической памяти LLM [2]. Для retrieval-задач широко используются bi-encoder эмбединги семейства Sentence-BERT [1]. При этом dense retrieval не всегда устойчив к exact-match сигналам, аббревиатурам и редким формулировкам, поэтому на практике его часто сравнивают или комбинируют с sparse-поиском семейства BM25 [3]. Для длинных документов важным оказывается не только выбор embedder, но и способ разбиения документа на фрагменты, потому что качество итогового ответа сильно зависит от того, какой именно кусок текста оказался в top- k выдаче.

4. Архитектура системы

Система состоит из трёх слоёв:

1. Подготовка данных. Документы из `RAG_CODE/rag/data/` конвертируются в `markdown`, после чего к ним применяется предобработка и чанкирование.
2. Retrieval-слой. Для чанков строятся векторные представления, после чего сравниваются `dense`, `BM25` и `ensemble`-стратегии поиска. Поверх кандидатов может применяться `reranking`.
3. Продуктовый интерфейс. Telegram-бот позволяет задавать вопрос, получать ответ и при необходимости видеть документы-источники.

В проекте использовались два основных метода чанкирования. **Plain recursive** разбивает весь документ как единый текст: на вход рекурсивному `splitter`'у подаётся полный `markdown`-файл, и он режется по стандартным разделителям до заданного лимита `chunk_size` с перекрытием `chunk_overlap`. **Header-aware recursive** работает в два шага: сначала документ делится по `markdown`-заголовкам или нумерованным секциям, затем каждая секция дочанкивается тем же `recursive splitter`'ом. Благодаря этому во втором режиме сохраняется привязка фрагмента к конкретному разделу документа, что важно для интерпретируемости ответа.

Для базовой конфигурации после предобработки и `header-aware` чанкирования корпус со-держал 348 чанков: 146 из ПОПАТКУС, 58 из ПВРО 2024 и 144 из устава.

5. Датасеты и метрики

5.1. Retrieval-набор

Основной `evaluation set` хранится в `experiments/question_pool.csv`. Он содержит 60 вопросов:

- 24 вопроса в подвыборке `validation`;
- 30 вопросов в подвыборке `test`;
- 6 пограничных вопросов в подвыборке `analysis`.

Автоматически оцениваются 54 вопроса, ещё 6 используются только для качественного разбора. В набор вошли вопросы по трём документам: ПОПАТКУС, ПВРО 2024 и `Ustav_01.02.2016-2`. Для каждого вопроса размечены ожидаемый документ, ожидаемый раздел, тип вопроса, сложность и режим оценки. Это позволяет считать как `doc-level`, так и `section-level` метрики. Расширение набора с 30 до 54 автоматически оцениваемых вопросов было важно для того, чтобы различия между конфигурациями не выглядели случайными.

5.2. Generation-набор

Для генерации был собран отдельный файл `experiments/generation_eval_set.csv` из 15 вопросов с краткими `reference answers`. Этот набор покрывает все три документа и нужен не для `retrieval`, а для оценки того, насколько итоговый ответ близок к эталону и насколько он `grounded` в извлечённом контексте.

5.3. Retrieval-метрики

В retrieval-экспериментах использовались:

- document Hit@1;
- document Hit@3;
- MRR;
- nDCG@3;
- section Hit@1 и section Hit@3.

Doc-level метрики на таком маленьком корпусе быстро достигают потолка, поэтому section-level метрики оказываются более чувствительными к качеству чанкирования, структуры индекса и reranking.

5.4. Generation-метрики

Для генерации использовались три автоматические метрики:

- ROUGE-L F1 по отношению к краткому эталонному ответу;
- grounding overlap --- доля содержательных токенов ответа, присутствующих в retrieval-контексте;
- citation rate --- доля ответов, в которых модель реально оставила ссылку на источник в требуемом формате.

Такой набор позволяет развести три свойства ответа: близость к reference answer, степень опоры на контекст и способность модели явно цитировать источник.

6. Эксперименты и результаты

6.1. Предобработка

Сравнивались два режима: raw и clean. В режиме clean удалялись markdown-артефакты, footnotes, boilerplate и служебные блоки.

Конфигурация	Hit@1	Hit@3	MRR	Sec@1	Sec@3
Raw + dense	1.0000	1.0000	1.0000	0.7917	0.8333
Clean + dense	1.0000	1.0000	1.0000	0.7917	0.8333

Таблица 1. Сравнение предобработки на validation

На данном корпусе очистка не изменила ни doc-level, ни section-level качество. Следовательно, её польза здесь состоит не в росте метрик, а в повышении воспроизводимости индекса и уменьшении служебного шума.

6.2. Размер чанка и метод чанкирования

В экспериментах по размеру чанка варьировались пары chunk_size/chunk_overlap: 768/128, 1024/256 и 1536/384. Во всех случаях использовался один и тот же embedder и

один и тот же dense retrieval-контур, чтобы изменение метрик объяснялось именно структурой индекса.

Размер чанка	Hit@1	Hit@3	MRR	Sec@1	Sec@3
768 / 128	0.9583	1.0000	0.9792	0.7500	0.8750
1024 / 256	1.0000	1.0000	1.0000	0.7917	0.8333
1536 / 384	0.9167	0.9583	0.9458	0.7083	0.7917

Таблица 2. Сравнение размеров чанка на validation

Под **методом чанкирования** в этой работе понимается не размер фрагмента, а логика его построения:

- **Plain recursive**: весь markdown-документ подаётся как единый текст, после чего рекурсивный splitter делит его на куски по стандартным разделителям без сохранения границ секций.
- **Header-aware recursive**: документ сначала делится по markdown-заголовкам или нумерованным разделам, а затем каждая секция отдельно режется recursive splitter'ом с тем же размером и overlap. В итоговом чанке сохраняется метадата исходного раздела.

Метод чанкирования	Hit@1	Hit@3	MRR	Sec@1	Sec@3
Plain recursive	1.0000	1.0000	1.0000	0.2500	0.2500
Header-aware recursive	1.0000	1.0000	1.0000	0.7917	0.8333

Таблица 3. Сравнение методов чанкирования на validation

Получилось важное наблюдение: doc-level метрики одинаковы, но plain recursive почти разрушает section-level интерпретируемость. Для нормативного чат-бота header-aware чанкирование существенно полезнее, потому что оно лучше сохраняет структуру источника и снижает смешивание соседних правил внутри одного фрагмента.

6.3. Сравнение embedders

Embedder	Hit@1	Hit@3	MRR	Sec@1	Sec@3
deepvk/USER-bge-m3	1.0000	1.0000	1.0000	0.7917	0.8333
intfloat/multilingual-e5-small	1.0000	1.0000	1.0000	0.7917	0.8333
paraphrase-multilingual-MiniLM-L12-v2	0.7917	0.9583	0.8833	0.5000	0.6250

Таблица 4. Сравнение embedders на validation

Наиболее слабый результат показал MiniLM, особенно на noisy и section-sensitive кейсах. deepvk/USER-bge-m3 и multilingual-e5-small дали одинаковый потолок по doc-level метрикам, поэтому в качестве основного embedder был оставлен deepvk/USER-bge-m3 как уже интегрированный в продуктовый контур.

Retriever	Hit@1	Hit@3	MRR	Sec@1	Sec@3
Dense	1.0000	1.0000	1.0000	0.7917	0.8333
BM25	0.9167	0.9583	0.9458	0.5000	0.7083
Ensemble (RRF)	1.0000	1.0000	1.0000	0.7083	0.8750
Ensemble + rerank	1.0000	1.0000	1.0000	0.7500	0.7917

Таблица 5. Сравнение retrieval-стратегий на validation

6.4. Dense, sparse, ensemble и reranking

BM25 уступил dense retrieval на текущем корпусе, но оказался полезен как дополнительный exact-match сигнал в ансамбле. Ensemble поднял section Hit@3 до 0.8750, однако не улучшил doc-level метрики относительно сильного dense baseline. Reranking в этой конфигурации не дал выигрыша и даже немного ухудшил section-level показатели, что можно объяснить маленьким корпусом и уже достигнутым потолком по doc-level качеству.

6.5. Финальная проверка на test

Лучшая по критерию простоты и doc-level качества конфигурация --- dense retrieval с `deprvk/USER-bge-m3`, header-aware чанкированием и размером чанка 1024/256 --- была проверена на отложенной test-подвыборке.

Конфигурация	Hit@1	Hit@3	MRR	Sec@1	Sec@3
Baseline dense on test	1.0000	1.0000	1.0000	0.7333	0.8333

Таблица 6. Финальная retrieval-оценка на test

Итог показывает, что retrieval-компонент надёжен на уровне правильного документа, но section-level точность остаётся заметно ниже единицы. Для нормативного чат-бота это важно, потому что пользователю нужен не просто правильный документ, а максимально точный раздел внутри него.

6.6. Generation

В generation-экспериментах сравнивались три режима:

- `extractive_top1`: ответ формируется из лучшего найденного фрагмента;
- `llm_strict`: короткий grounded-ответ без обязательных ссылок;
- `llm_cited`: grounded-ответ с требованием явных ссылок на источник.

Режим генерации	Failed	ROUGE-L F1	Grounding	Citation rate
Extractive top-1	0	0.1854	0.9505	1.0000
LLM strict	0	0.4091	0.7987	0.0000
LLM cited	0	0.1800	0.8645	0.8000

Таблица 7. Generation-оценка на наборе из 15 вопросов

Extractive baseline оказался самым grounded, но самым слабым по качеству формулировки. Режим `llm_strict` показал лучшую близость к `reference answers`, то есть лучше суммировал найденный контекст. Режим `llm_cited` действительно научился оставлять ссылки в большинстве ответов, но потерял часть качества по ROUGE-L. Для продукта это означает, что строгая генерация без принудительного `citation-format` пока выглядит лучшим компромиссом, а `citation-mode` полезен как более прозрачный, но и более тяжёлый режим.

6.7. Pilot AgenticRAG

Дополнительно был реализован `pilot AgenticRAG`-контур для 6 пограничных вопросов из `split analysis`. Агент порождал 2--3 поисковых подзапроса, выполнял `retrieval` по каждому из них, затем объединял кандидатов через `reciprocal rank fusion` и формировал ответ с указанием источников.

Качественный анализ показал смешанный результат:

- на вопросах про нарушения во время экзамена и неявку на аттестацию агент действительно расширял охват контекста и подтягивал фрагменты из нескольких разделов;
- на вопросе про шпаргалки агент сгенерировал нерелевантный подзапрос про другой университет, что ухудшило `retrieval`;
- в целом `AgenticRAG` повысил покрытие неоднозначных кейсов, но добавил новую точку отказа --- качество самих подзапросов.

Следовательно, `agentic`-подход в данной задаче выглядит перспективным именно для `boundary`-вопросов, но пока не должен заменять базовый `retrieval`-контур.

7. Обсуждение результатов

С точки зрения бизнес-целей проект можно считать успешным. Для типовых вопросов по трём документам система стабильно находит правильный документ, а `generation`-компонент способен формировать пригодный для пользователя краткий ответ. С точки зрения технических целей также удалось выполнить ключевые пункты: `retrieval` и `generation` оцениваются отдельно, есть воспроизводимый экспериментальный контур, датасеты описаны и расширены, а различия между конфигурациями представлены в табличном виде.

Однако у работы есть и ограничения. Во-первых, `doc-level` метрики уже упираются в потолок, поэтому для следующего этапа важнее наращивать сложность корпуса, количество документов и `multi-document` сценарии, чем ещё раз сравнивать сильные `dense`-модели на тех же трёх источниках. Во-вторых, `generation`-метрики остаются эвристическими: они полезны для быстрой автоматической проверки, но не заменяют экспертную ручную валидацию фактуальности.

8. Заключение

В работе была реализована и оценена `retrieval-augmented` система для вопросов по нормативным документам НИУ ВШЭ. В отличие от промежуточной версии проекта, итоговый

контур включает не только базовый dense retrieval, но и сравнение предобработки, размеров и методов чанкирования, нескольких embedders, dense/sparse/ensemble retrieval, reranking, generation и pilot AgenticRAG.

Основные выводы следующие:

1. На текущем корпусе dense retrieval с deepvk/USER-bge-m3 и header-aware чанкированием уже достигает потолка по doc-level метрикам.
2. Размер чанка влияет на качество: конфигурация 1024/256 оказалась лучшей среди протестированных.
3. Plain recursive чанкирование непригодно для задач, где важна интерпретируемость и привязка к разделу.
4. BM25 полезен как дополнительный сигнал, но не превосходит сильный dense baseline на данном корпусе.
5. В generation extractive baseline даёт максимальную groundedness, а LLM strict --- лучший компромисс между качеством формулировки и привязкой к контексту.
6. AgenticRAG потенциально полезен на boundary-вопросах, но требует более надёжной генерации подзапросов.

9. Перспективы дальнейшей разработки

Следующим этапом развития проекта должно стать не только количественное расширение корпуса, но и усложнение самих пользовательских сценариев. Текущая версия системы хорошо работает на типовых вопросах по трём документам, однако дальнейшее развитие должно быть направлено на повышение устойчивости retrieval в multi-document и неоднозначных кейсах, а также на более строгую валидацию generation-компонента.

Ключевые направления дальнейшей разработки:

- расширить корпус документов и gold set;
- добавить более сложные multi-hop и multi-document вопросы;
- провести ручную экспертную оценку generation;
- исследовать более сильные multilingual rerankers;
- доработать AgenticRAG и при необходимости сравнить его с graph-based представлением корпуса.

10. Использование ИИ

При подготовке проекта и отчёта использовались инструменты искусственного интеллекта, в первую очередь ChatGPT/Codex от OpenAI. Они применялись в двух ролях. Во-первых, как вспомогательный инструмент при работе с кодом: для рефакторинга экспериментальных скриптов, поиска и исправления ошибок, а также помощи в воспроизведении retrieval- и generation-экспериментов. Во-вторых, как инструмент редакторской поддержки: для улучшения структуры текста, формулировок и устранения технических проблем при сборке L^AT_EX-документа.

При этом постановка исследовательской задачи, формирование датасетов, выбор метрик,

дизайн экспериментов, проверка и интерпретация результатов, а также итоговые выводы были выполнены мной самостоятельно и дополнительно вручную проверены перед включением в финальную версию отчёта.

Список литературы

- [1] N. Reimers, I. Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of EMNLP-IJCNLP*, 2019.
- [2] P. Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, 2020.
- [3] S. Robertson, S. Walker. Some simple effective approximations to the 2-Poisson model for probabilistic weighted retrieval. In *Proceedings of SIGIR*, 1994.
- [4] G. Izacard et al. Atlas: Few-shot Learning with Retrieval Augmented Language Models. *Journal of Machine Learning Research*, 2023.